



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution Affiliated to Anna University Chennai - Approved by AICTE - Accredited by NAAC with "A+" Grade

SATHYAMANGALAM - 638 401 ERODE DISTRICT TAMIL NADU INDIA

Ph: 04295-226000 / 221289 Fax: 04295-226666 E-mail: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

22AI503/22AM503 MACHINE LEARNING

UNIT III – CLASSIFICATION

Decision Trees ID3, C4.5, and CART

1. Introduction

1.1 Definition

A Decision Tree is a supervised, eager-learning classifier (and regressor) that recursively partitions the feature space into regions, represented as a tree of decision nodes (attribute tests) and leaf nodes (class predictions). ID3, C4.5, and CART are the three foundational decision-tree induction algorithms, each refining the splitting criterion and capabilities of its predecessor.

Definition:

Decision Tree: a flowchart-like model where internal nodes test an attribute, branches represent outcomes, and leaves assign a class label or numeric value.

ID3 (Iterative Dichotomiser 3): the original algorithm, using Information Gain (entropy reduction) for categorical attributes.

C4.5: ID3's successor, adding Gain Ratio, continuous attribute handling, missing-value support, and pruning.

CART (Classification and Regression Trees): introduces Gini Index, strictly binary splits, and native support for both classification and regression — the basis of most modern implementations (including scikit-learn).

1.2 Importance in Machine Learning

Decision trees are among the most interpretable ML models — their IF-THEN rule structure is directly human-readable, making them valuable in regulated domains (finance, healthcare) requiring explainable decisions. They also form the foundational building block for powerful ensemble methods: Random Forests (bagging of trees) and Gradient Boosting (sequential boosting of trees) — understanding individual trees is prerequisite to understanding these ensembles.

1.3 Role in the Classification Pipeline

Decision trees require minimal preprocessing (no feature scaling needed, natively handle mixed categorical/numeric data) making them a fast, low-effort baseline. They are also commonly used for feature importance ranking (informing feature selection for other models) before being superseded by ensemble variants for final production accuracy.

2. Learning Outcomes

Understand

- The recursive partitioning algorithm shared by ID3, C4.5, and CART
- Entropy, Information Gain, Gain Ratio, and Gini Index as splitting criteria

- The key differences and improvements across ID3 to C4.5 to CART

Apply

- Manually compute entropy, information gain, and Gini index for a small dataset
- Build decision tree classifiers using scikit-learn (CART-based) with appropriate hyperparameters
- Apply pre-pruning (max_depth, min_samples_split) and post-pruning (cost-complexity) to control overfitting

Analyze

- Diagnose overfitting from an unpruned, deeply grown decision tree
- Evaluate the high-cardinality attribute bias in Information Gain and how Gain Ratio addresses it
- Compare ID3, C4.5, and CART for a given dataset and task (classification vs regression, attribute types present)

3. Detailed Technical Concepts

3.1 Decision Tree Structure and Induction

A decision tree is built top-down via recursive partitioning: starting at the root with the full training set, the algorithm selects the attribute (and, for continuous attributes, a threshold) that best splits the data according to a chosen impurity/splitting criterion, creates child nodes for each resulting subset, and recurses on each child until a stopping condition is met (pure node, no remaining attributes, or a depth/size limit).

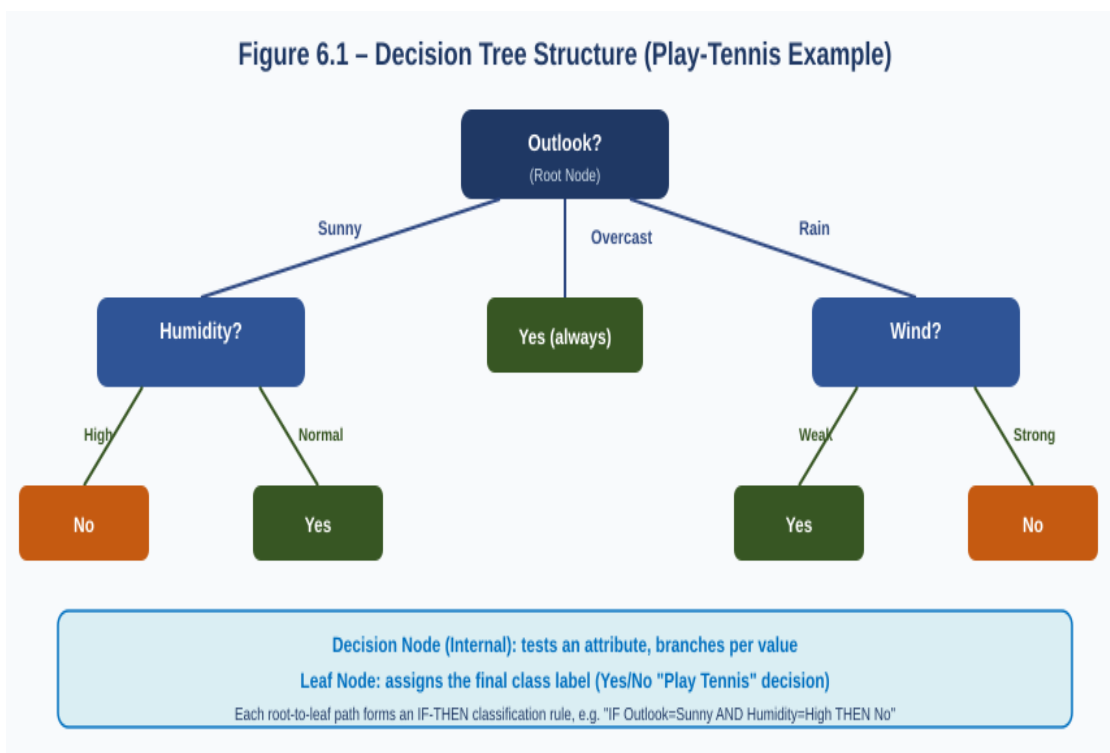


Figure 6.1 – Decision Tree Structure (Play-Tennis Example)

3.2 Splitting Criteria

3.2.1 Entropy

$H(S) = -\sum_i (p_i \cdot \log_2(p_i))$, where p_i is the proportion of class i in set S . Entropy measures the impurity/disorder of a node: 0 for a pure node (single class), maximised (1.0 for binary) at a perfectly even class split.

3.2.2 Information Gain (ID3)

$IG(S,A) = H(S) - \sum_v (|S_v|/|S| \cdot H(S_v))$, the reduction in entropy achieved by splitting set S on attribute A into subsets S_v . ID3 selects, at each node, the attribute that maximises Information Gain.

Bias: Information Gain inherently favours attributes with many distinct values (e.g., a unique customer ID would create a perfectly pure split for every single record, yielding maximum but meaningless information gain) — this is a key limitation that C4.5 addresses.

3.2.3 Gain Ratio (C4.5)

$GainRatio(S,A) = IG(S,A) / SplitInfo(S,A)$, where $SplitInfo(S,A) = -\sum_v (|S_v|/|S| \cdot \log_2(|S_v|/|S|))$. SplitInfo measures the intrinsic information of the split itself (how many subsets it creates and their relative sizes), penalising attributes that fragment the data into many small subsets. C4.5 selects the attribute maximising Gain Ratio, correcting ID3's high-cardinality bias.

3.2.4 Gini Index (CART)

$Gini(S) = 1 - \sum_i (p_i^2)$, measuring the probability that a randomly chosen element would be incorrectly classified if randomly labelled according to the class distribution in S . CART selects the split minimising the weighted Gini Index of the resulting children. Computationally cheaper than entropy (no logarithm), and tends to isolate the most frequent class in its own branch.

Figure 6.2 – Splitting Criteria: Entropy, Gini Index, and Information Gain

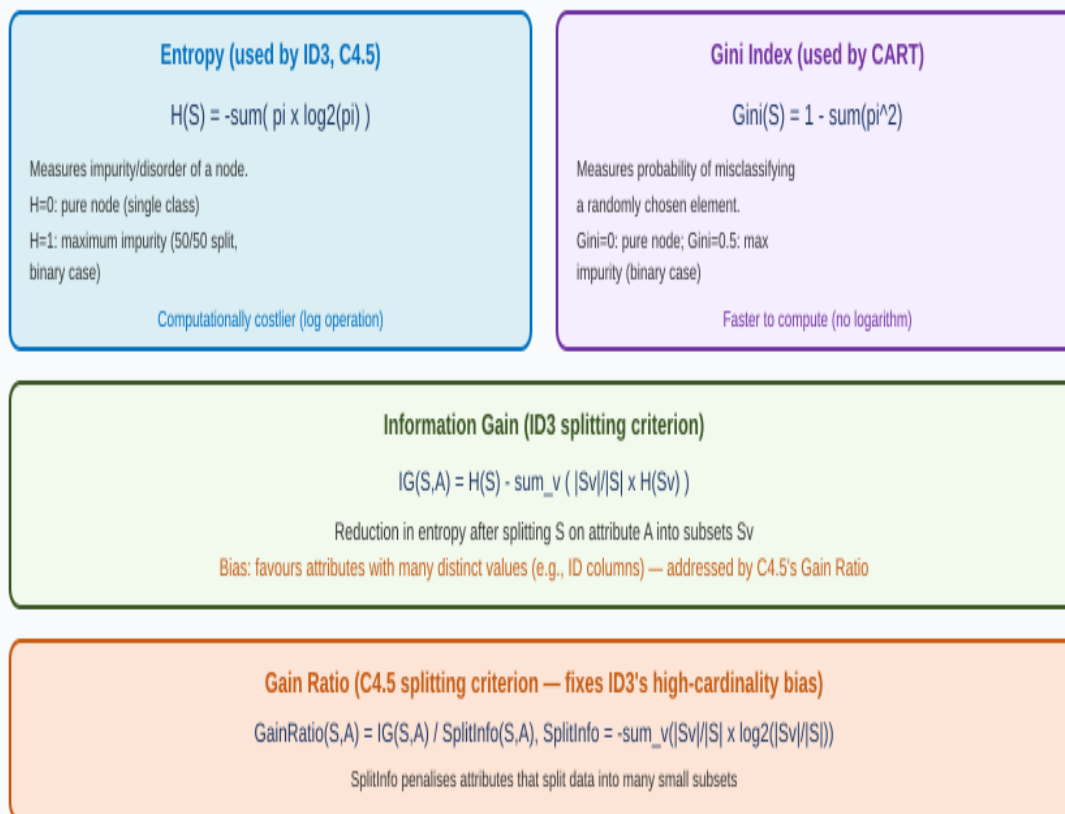


Figure 6.2 – Splitting Criteria: Entropy, Gini Index, Information Gain, and Gain Ratio

3.3 ID3 (Iterative Dichotomiser 3)

Developed by Ross Quinlan (1986), ID3 was the first widely-used decision tree algorithm. It uses Information Gain as its splitting criterion, builds multi-way trees (one branch per distinct attribute value), and works only with categorical attributes — continuous attributes must be discretized beforehand. ID3 has no built-in pruning mechanism, making it prone to overfitting, and offers no native handling of missing values.

3.4 C4.5

Also developed by Quinlan (1993) as a direct successor to ID3, C4.5 introduces several critical improvements:

- Gain Ratio replaces Information Gain, correcting the high-cardinality attribute bias.
- Continuous attribute handling: finds an optimal binary threshold split (e.g., Age ≤ 35 vs Age > 35) for numeric attributes, rather than requiring pre-discretization.
- Missing value handling: during training, instances with missing values for the splitting attribute are probabilistically distributed across child branches; during prediction, multiple paths are explored and weighted.
- Pruning: applies pessimistic error pruning (post-pruning) to reduce overfitting, removing or collapsing subtrees that do not improve estimated true error.
- Rule generation: can convert the tree into a set of human-readable IF-THEN rules, with further per-rule simplification.

C4.5 became the de facto standard decision tree algorithm for over a decade and is implemented as the J48 algorithm in WEKA.

3.5 CART (Classification and Regression Trees)

Developed by Breiman, Friedman, Olshen, and Stone (1984), CART introduces several distinguishing characteristics that make it the basis for most modern implementations:

- Strictly binary splits: every node splits into exactly two children, regardless of attribute type or number of distinct values — simplifying the tree structure and enabling consistent treatment of categorical and continuous attributes.
- Gini Index for classification: the default impurity measure (entropy can also optionally be used in many CART implementations).
- Regression support: for continuous targets, CART uses variance reduction (minimising weighted sum of squared errors within child nodes) as the splitting criterion — extending tree-based modelling beyond classification into regression.
- Cost-complexity pruning: a principled post-pruning method that balances tree size against training error using a complexity parameter α , selected via cross-validation.
- Surrogate splits: handles missing values by finding alternative ("surrogate") attributes that produce similar splits to the primary chosen attribute.

CART is the algorithm underlying scikit-learn's `DecisionTreeClassifier` and `DecisionTreeRegressor`, as well as the base learner in Random Forests and Gradient Boosted Trees.

Figure 6.3 – ID3 vs C4.5 vs CART Comparison

ID3 (1986)	C4.5 (1993)	CART (1984)
Splitting Criterion: Information Gain (entropy-based)	Splitting Criterion: Gain Ratio (normalised IG)	Splitting Criterion: Gini Index (classification), Variance reduction (regression)
Attribute Types: Categorical only (no numeric)	Attribute Types: Categorical AND continuous (via threshold splits)	Attribute Types: Categorical AND continuous
Tree Type: Multi-way splits (one branch per attribute value)	Tree Type: Multi-way (categorical), binary (continuous)	Tree Type: STRICTLY binary splits only
Missing Values: Not natively handled	Missing Values: Handled via probabilistic split	Missing Values: Surrogate splits
Pruning: No built-in pruning (overfits)	Pruning: Pessimistic error pruning (built-in)	Pruning: Cost-complexity pruning
Key Weakness: Biased toward high-cardinality attributes (e.g., unique IDs)	Improvement: fixes ID3's cardinality bias and overfitting	Key Feature: supports BOTH classification AND regression
Use: Educational/historical; superseded by C4.5 in practice	Use: Classic, well-rounded; basis for J48 in WEKA	Use: Industry standard; sklearn DecisionTreeClassifier default

Figure 6.3 – ID3 vs C4.5 vs CART Comparison

3.6 Pruning Strategies

3.6.1 Pre-Pruning (Early Stopping)

Stops tree growth before it perfectly fits the training data, using constraints such as: `max_depth` (maximum tree depth), `min_samples_split` (minimum samples required to split a node), `min_samples_leaf` (minimum samples required at a leaf), or a minimum information gain/impurity decrease threshold. Computationally cheap but risks stopping too early ("horizon effect" — missing a useful split that would have appeared one level deeper).

3.6.2 Post-Pruning (Pruning After Full Growth)

Grows the full tree first, then removes/collapses subtrees that do not improve validation performance. C4.5 uses pessimistic error pruning (statistically estimating whether removing a subtree increases true error); CART uses cost-complexity pruning (minimising `training_error + alpha * num_leaves`, with `alpha` selected via cross-validation — scikit-learn: `DecisionTreeClassifier(ccp_alpha=...)`). Generally more reliable than pre-pruning since it considers the fully-grown tree's structure, at the cost of additional computation.

3.7 Worked Example — Computing Information Gain

For the Play-Tennis dataset (9 Yes, 5 No out of 14 records): $H(S) = -(9/14) \cdot \log_2(9/14) - (5/14) \cdot \log_2(5/14) = 0.940$. Splitting on Outlook (Sunny: 2Y/3N, Overcast: 4Y/0N, Rain: 3Y/2N) gives weighted child entropy = $(5/14) \cdot 0.971 + (4/14) \cdot 0 + (5/14) \cdot 0.971 = 0.694$, so $IG(S, \text{Outlook}) = 0.940 - 0.694 = 0.246$ — the highest information gain among all candidate attributes, making Outlook the root split in ID3/C4.5.

4. Architecture / Decision Tree Induction Workflow

Step	Action	Method / Code	Output
1	Compute impurity of current node	Entropy / Gini Index of the node's class distribution	Baseline impurity score
2	Evaluate all candidate splits	For each attribute (and threshold, if continuous)	Candidate split impurity reductions
3	Select best split	Max Information Gain (ID3) / Gain Ratio (C4.5) / Min Gini (CART)	Chosen splitting attribute/threshold
4	Partition data, create child nodes	Recursively repeat steps 1-3 on each subset	Grown subtree
5	Check stopping criteria	Pure node, max_depth, min_samples, no attributes left	Decision to stop or continue recursion
6	Apply pruning	Pre-pruning (hyperparameters) or post-pruning (cost-complexity)	Final pruned tree
7	Validate on held-out test set	accuracy_score, confusion_matrix	Final performance metrics

5. Industry Tools and Technologies

5.1 Commercial Tools

Tool	Vendor	Decision Tree Algorithm Support
SPSS Modeler	IBM	C5.0 (C4.5 successor), CHAID, CART
SAS Enterprise Miner	SAS	PROC HPSPLIT supporting CHAID, CART, C4.5-style trees
RapidMiner	RapidMiner	Decision Tree operator with Gini/Gain Ratio/Info Gain criteria
KNIME	KNIME	Decision Tree Learner node (C4.5-based)
Weka	Univ. of Waikato	J48 (C4.5 implementation), SimpleCart

5.2 Open-Source Tools

- scikit-learn: DecisionTreeClassifier/Regressor (CART-based, optimised implementation)
- rpart (R): CART implementation, the standard R decision tree package
- C50 (R): implementation of Quinlan's C5.0 algorithm (C4.5's commercial successor)
- xgboost / lightgbm / catboost: gradient-boosted tree ensembles built on CART-style base learners
- graphviz / dtreeviz: visualisation libraries for inspecting and interpreting trained decision trees

6. Real-World Applications

Domain	Application	Algorithm Used	Why This Algorithm
Credit Scoring	Loan approval decision rules	C4.5 / CART	Interpretable IF-THEN rules for regulatory compliance
Healthcare	Clinical diagnosis decision support	CART	Handles mixed categorical/numeric vitals, supports missing data
Marketing	Customer segmentation (CHAID-related)	CART	Binary splits give clean, explainable segments
Manufacturing	Predicting equipment failure (regression)	CART (regression mode)	Native support for continuous target prediction
Ensemble Learning	Base learner for Random Forest/XGBoost	CART	Fast, low-bias base learner ideal for bagging/boosting

7. Advantages and Limitations

Advantages	Limitations
Highly interpretable — directly readable as IF-THEN rules	Prone to overfitting without pruning, especially with deep trees
No feature scaling required (unlike kNN, SVM)	High variance — small data changes can produce very different trees
Natively handles mixed categorical and numeric data (C4.5, CART)	Information Gain (ID3) is biased toward high-cardinality attributes
CART supports both classification and regression	Greedy, locally-optimal splitting may not find the globally optimal tree
Forms the foundation for powerful ensemble methods (RF, GBM)	Single trees often underperform ensembles on complex datasets

8. Case Study

CASE STUDY

Case: C4.5 in the Top 10 Data Mining Algorithms (Wu et al., 2008)

Problem: In 2006, the IEEE International Conference on Data Mining (ICDM) convened a panel of leading researchers to identify the most influential data mining algorithms in the field's history.

Recognition: C4.5 was ranked #1 on the resulting "Top 10 Algorithms in Data Mining" list — ahead of k-Means, SVM, Apriori, EM, PageRank, AdaBoost, kNN, Naive Bayes, and CART.

Rationale for Ranking: The panel highlighted C4.5's combination of strong empirical performance, computational efficiency, and crucially, interpretability — its IF-THEN rule output remains directly understandable to domain experts without machine learning background, a property few competing algorithms of the era matched.

Industrial Impact: C4.5 (and its commercial successor C5.0) became embedded in numerous commercial data mining suites (SPSS Clementine/Modeler, SAS Enterprise Miner) and remains the algorithm underlying WEKA's widely-used J48 classifier, used in thousands of academic and industrial projects since the 1990s.

Conclusion: C4.5's recognition as the most influential data mining algorithm demonstrates that for many practical applications, interpretability and reliability can outweigh raw predictive accuracy — and that solving the right combination of problems (categorical+continuous attributes, missing values, overfitting via pruning) at the right time can have an outsized, lasting impact on a field.

9. Scenario-Based Learning

SCENARIO 1

SCENARIO 1: ID3 Overfits Due to a High-Cardinality ID Column

Background: A junior data scientist builds an ID3-style decision tree (using raw Information Gain) to predict customer churn, accidentally including a `customer_id` column alongside legitimate features (`tenure`, `monthly_charges`, `contract_type`).

Technical Environment: Python, custom ID3 implementation, 10,000 customer records.

Problem Statement: The tree selects `customer_id` as the ROOT split (since every customer has a unique ID, this split achieves perfect, maximal information gain by creating one pure leaf per training record), resulting in 100% training accuracy but only 52% test accuracy (essentially random for this near-balanced problem).

Available Evidence: $IG(S, \text{customer_id}) = H(S) - 0 = H(S)$ (maximum possible, since every resulting subset is pure with exactly 1 record), far exceeding IG for any meaningful feature like `contract_type` ($IG=0.18$) or `tenure` ($IG=0.21$).

Investigation: Recognise the classic high-cardinality attribute bias inherent to raw Information Gain — ID3's splitting criterion has no mechanism to penalise attributes that trivially fragment the data.

Analysis: Two fixes: (1) remove identifier columns from the feature set entirely (they should never be model inputs), and (2) switch to C4.5's Gain Ratio, which would naturally down-weight `customer_id` due to its very high `SplitInfo`.

Findings: After removing `customer_id` and switching to Gain Ratio-based splitting (C4.5-style), test accuracy improved to 79%, with `contract_type` correctly identified as the most informative root-level split.

Recommended Solution: Always remove unique identifier columns before training any decision tree.

Additionally, prefer C4.5 (Gain Ratio) or CART (Gini Index) over raw ID3 (Information Gain) specifically because they are far more robust to high-cardinality attribute bias.

SCENARIO 2

SCENARIO 2: Unpruned CART Tree Overfits Training Data

Background: A retail analytics team trains a CART decision tree (scikit-learn `DecisionTreeClassifier` with default settings) to predict whether a customer will respond to a marketing campaign, using 25 behavioural features.

Technical Environment: Python, scikit-learn, 50,000 customer records, 80/20 train/test split.

Problem Statement: The unpruned tree achieves 99.97% training accuracy but only 68% test accuracy — a massive 32-point generalisation gap, indicating severe overfitting.

Available Evidence: `model.get_depth()` reports a tree depth of 38 with 4,200+ leaf nodes — far deeper and more complex than the underlying signal likely warrants, with many leaves containing just 1-2 training samples.

Investigation: Plot training vs validation accuracy across a range of `max_depth` values (1 to 30) to identify where validation accuracy plateaus or starts declining while training accuracy keeps climbing.

Analysis: Validation accuracy peaks around `max_depth=8` (82% validation accuracy) and then declines as depth increases further, while training accuracy continues climbing toward 100% — the classic overfitting signature.

Findings: Constraining the tree with `max_depth=8`, `min_samples_leaf=20` reduced the tree to 95 leaves;

training accuracy dropped to 86% but test accuracy improved dramatically to 81% — a far smaller, more trustworthy generalisation gap. Cost-complexity pruning (ccp_alpha tuned via cross-validation) achieved a similar result (80% test accuracy) with an even more compact tree (62 leaves).
Recommended Solution: Never deploy an unpruned decision tree. Always apply either pre-pruning hyperparameters (max_depth, min_samples_leaf, min_samples_split) or post-pruning (cost-complexity pruning via ccp_alpha), validated through cross-validation, before considering a tree production-ready.

SCENARIO 3

SCENARIO 3: Missing Values Mishandled in a Healthcare Dataset

Background: A hospital builds a CART-based decision tree to predict diabetic risk using 12 clinical features, several of which (e.g., insulin_level, skin_thickness) have substantial missing values (15-35% missing) due to inconsistent historical test ordering.

Technical Environment: Python, scikit-learn DecisionTreeClassifier (which does NOT natively support missing values, unlike some CART/C4.5 implementations), 768 patient records.

Problem Statement: The team initially used simple mean imputation for all missing values before training, achieving 74% test accuracy — but a domain expert flagged that "missingness itself" (whether a test was ordered at all) was suspected to correlate with disease severity (sicker patients get more tests ordered).

Available Evidence: Comparing diabetic rate between patients with missing insulin_level (41% diabetic) vs complete insulin_level (31% diabetic) confirms missingness is NOT random (MNAR/MAR) — it carries diagnostic signal.

Investigation: Mean imputation discards this "missingness as signal" information entirely, replacing it with an uninformative average value.

Analysis: Add explicit "was_missing" binary indicator features (e.g., insulin_level_was_missing) alongside the (still mean-imputed) original features, allowing CART to split on missingness itself if informative.

Findings: With added missingness indicators, test accuracy improved from 74% to 80%; feature importance ranking showed insulin_level_was_missing as the 4th most important feature overall — confirming the domain expert's hypothesis and substantially improving the model.

Recommended Solution: When missing values may be informative (MAR/MNAR, not pure MCAR), add explicit missingness indicator features before training decision trees that lack native missing-value handling. For implementations with native missing-value support (e.g., C4.5's probabilistic distribution or CART's surrogate splits), evaluate whether the native mechanism adequately captures this signal, or whether explicit indicators still add value.

10. Common Errors and Troubleshooting

Error	Cause	Impact	Fix
Including identifier columns as features	Not excluding unique-value columns before training	High-cardinality bias dominates split selection (esp. ID3)	Remove ID/unique-identifier columns before training
Training without any pruning	Using default unconstrained tree growth	Severe overfitting; large train/test accuracy gap	Apply max_depth/min_samples_leaf (pre-pruning) or ccp_alpha (post-pruning)
Using raw Information Gain (ID3-style) on high-cardinality data	Not aware of the bias	Misleading "best" splits that do not generalise	Use Gain Ratio (C4.5) or Gini Index (CART) instead
Mean-imputing informative missing values	Treating all missingness as MCAR	Loses predictive signal carried by missingness pattern	Add missingness indicator features when MAR/MNAR is suspected

Comparing a single tree to ensemble accuracy and concluding trees are "bad"	Unfair comparison without considering ensembles' purpose	Premature dismissal of decision trees as a technique	Recognise single trees as interpretable baselines/building-blocks; use Random Forest/GBM when raw accuracy is the priority
---	--	--	--

11. Key Terminologies

Term	Definition
Decision Node	An internal tree node that tests an attribute and branches based on the result
Leaf Node	A terminal tree node assigning a final class label or numeric prediction
Entropy	Measure of impurity/disorder: $H(S) = -\sum(\pi_i \log_2(\pi_i))$; 0=pure, max at even split
Information Gain (IG)	Reduction in entropy from splitting on an attribute; ID3's splitting criterion
Gain Ratio	Information Gain normalised by Split Information; C4.5's splitting criterion, corrects high-cardinality bias
Gini Index	Probability of misclassification measure: $Gini(S) = 1 - \sum(\pi_i^2)$; CART's splitting criterion
Split Information	Intrinsic information of a split itself, used to normalise Information Gain into Gain Ratio
Pre-Pruning	Stopping tree growth early via constraints (max_depth, min_samples_split, etc.)
Post-Pruning	Growing the full tree then removing subtrees that do not improve validation performance
Cost-Complexity Pruning	CART's post-pruning method balancing training error against tree size via parameter alpha
Surrogate Split	CART's mechanism for handling missing values using alternative correlated attributes
Multi-way Split	A node splitting into one branch per distinct attribute value (ID3, C4.5 for categoricals)
Binary Split	A node splitting into exactly two branches (CART, always; C4.5 for continuous attributes)
Overfitting (in trees)	When a tree grows so deep it memorises training noise, harming generalisation

12. Topic Summary

UNIT III – Topic 6

DECISION TREES — ID3, C4.5, AND CART — SUMMARY

- Decision trees recursively partition the feature space via top-down, greedy attribute selection, forming an interpretable IF-THEN rule structure.
- ID3: uses Information Gain (entropy reduction), categorical attributes only, multi-way splits, no pruning, no missing-value handling — biased toward high-cardinality attributes.
- C4.5: uses Gain Ratio (fixes ID3's bias), handles continuous attributes via threshold splits, handles missing values probabilistically, includes pessimistic error pruning.

- CART: uses Gini Index (classification) or variance reduction (regression), strictly binary splits, cost-complexity pruning, surrogate splits for missing values — basis for scikit-learn and most modern implementations.
- Splitting criteria comparison: Entropy/IG (ID3) costly and biased; Gain Ratio (C4.5) corrects bias; Gini Index (CART) computationally cheaper, similar splits in practice to entropy.
- Pruning is essential: pre-pruning (max_depth, min_samples_leaf) is cheap but may stop early; post-pruning (cost-complexity, ccp_alpha) is more reliable but costlier.
- Decision trees require no feature scaling and natively handle mixed attribute types (C4.5, CART) — but are high-variance and prone to overfitting without pruning.
- Trees are the foundational building block for Random Forest (bagging) and Gradient Boosting (sequential boosting) ensembles.
- Tools: scikit-learn (DecisionTreeClassifier/Regressor — CART-based), WEKA (J48 — C4.5), R (rpart — CART, C50 — C4.5 successor).